

自动驾驶单次运行通用实时性—物理安全 双向诊断框架

Event + Loop · R/A/G/C/L · Dual Contracts · Bidirectional Diagnosis

适用系统 CARLA 0.9.15 + Apollo 10.0.0 + Bridge

分析范围 一次 run 内的事件响应、多源一致性、连续控制环、双层契约与物理损失

证据约束 Observed / Requirement / Model / Retrospective 严格分离

版本日期 2026-08-13

本方法只针对单次运行；不包含多 run 组间比较方法

摘要

本框架的分析对象不是“整个 run 的一个平均延迟”，也不是“一个已知注入故障”。一次 run 内并行建立两类基本分析单位：

$$U_E=(e_k,B_b,k,P_c,k,l_n,c,k)$$

$$U_L=(_q,W_r,m_s,_r)$$

其中，U_E 是 Event-Centric Unit：e_k 为 Safety-Relevant Event，B_b,k 为联合输入依赖束，P_c,k 为束内 source-to-effect path，l_n,c,k 为实际 job/chain instance；U_L 是 Loop-Centric Unit：_q 为控制环，W_r 为分析窗口，m_s 为驾驶/控制模式，_r 为速度、曲率、道路、目标密度、控制增益等 context。前者回答“事件响应是否及时”，后者回答“即使没有离散 hazard，连续闭环是否因采样、反馈或执行时间不规则而失去时间正确性”。run 只是原始数据和运行条件的容器。

对两类单位统一分析五维时间正确性：

$$T=[R,A,G,C,L]$$

- R：Reaction/Response；
- A：Age/Freshness；
- G：Fresh Update Gap/Continuity；
- C：Cross-Source Temporal Coherence；
- L：Closed-Loop Timing Stability。

其中 C 不是“另一个 Age”：它刻画同一次 Fusion/Planning/Control join 所消费的多源状态是否代表足够一致的世界时刻。L 也不是“某次最大 latency”：它刻画控制窗口内采样间隔、反馈延迟、actuation delay、phase drift、命令保持与物理跟踪误差之间的连续闭环关系。

六层不再是必须从 L1 顺序走到 L6 的流水线，而是六个相互约束的证据域：

证据域	新定位	必须回答的问题
L1 Temporal Stressor / Root-Cause	时间压力与根因候选域	什么机制制造了时间压力？
L2 Local Temporal Degradation	局部时间症状域	时间消耗在哪个 task、edge 或 sample-and-hold 环节？
L3 Cause-Effect / Loop Timing	端到端时间行为域	事件如何传播？联合输入是否时间一致？闭环采样、反馈、actuation 与跟踪是否稳定？
L4 Architectural + Physical Contracts	实时性判定枢纽	是否违反系统工程设计？是否进一步越过当前物理世界允许的安全时间？
L5 Physical Budget Loss	时间到物理安全的映射域	时间消耗占用了多少响应距离、分离距离、制动空间和安全裕量？
L6 Physical Safety State	物理后果域	最终是安全、低裕量、近失效、接触还是碰撞？

六层是六个 Evidence Domains，不是所有 run 都必须从 L1 顺序走到 L6 的固定 pipeline。分析有三种入口：

事件诊断：Event/C4/L6 seed -> L3 event lineage -> L2 local symptom -> L1 hypotheses

闭环诊断：Loop/C4-A/physical-behavior seed -> L3 loop window -> L2 jitter/delay primitive -> L1 hypotheses

后果方向：L4 contract state -> L5 physical budget loss -> L6 physical outcome

注入验证：L1 verified intervention -> L2 -> L3 -> L4 -> L5 -> L6 -> attribution

最终每个异常事件或闭环窗口必须回答六项：

1. 哪里：哪条 chain、哪个 task/edge/actuation segment 出现问题；
2. 性质：execution、queue、activation/phase、transport、staleness、reuse、gap、overwrite/drop、actuation，还是时钟/功能/物理替代解释；
3. 何时失去时间正确性：先区分 architectural contract miss 与 physical contract miss；只有 Assurance Extension 资格满足时，才进一步回答 guarantee loss；
4. 为什么：给出带证据、反证和可区分检验的根因候选，而不是把下游碰撞倒推成上游根因；

5. 损失多少：分别报告墙钟响应距离、requirement-constrained deadline-excess distance debt、实际几何/接触结果、基于要求阈值导出的安全裕量和模型预测的反事实裕量损失；
6. 结论有多强：明确是直接观测、系统级关联、段级定位、机制支持，还是 SUT 内生实时缺陷。

1. 框架边界与基本原则

1.1 本框架要诊断的“实时性问题”

实时性问题不是“某模块运行得比较慢”的同义词。只有当时间行为与明确的时间参考或安全时间契约发生关系时，才能作相应判断：

- 局部时间退化：局部等待、执行、数据年龄、更新间隔、输入时间偏斜或环周期/反馈延迟相对已声明参考发生异常；
- 工程时间正确性失效：本次观测的 task/edge/path/bundle/loop 时间行为越过合格 Architectural Temporal Contract；
- 安全关键时间正确性失效：本次观测时间行为进一步越过合格 Physical Dynamic Contract；
- 时间保证丢失：即使本次尚未 miss，合格的上界分析已经不能保证在契约内完成；
- 时间主导的物理安全失效候选：时间失效经物理预算传播并造成安全退化，同时重要功能、目标、时钟、相位和物理替代解释已被排除或界定；
- SUT 内生实时缺陷：有可重复、可隔离的 Apollo/Bridge 内部执行、调度、通信或数据机制证据。单次 run 通常只能支持到“候选”或“段级定位”，不能自动证明内生缺陷。

仅有高 latency、一次最大 gap、碰撞、制动晚或注入配置，均不能单独证明上述更强结论。

1.2 Observed Diagnostic Core 与 Assurance Extension 必须分开

Observed Diagnostic Core 对任何可解析 run 都执行，输出实际时间行为、工程/物理契约状态、症状定位和物理后果。它不因缺少 WCET/WCRT 而失败。

Assurance/Guarantee Extension

仅在部署模型与上界资格满足时执行，用于回答所有允许行为是否可保证满足契约。它需要 WCET/WCRT、scheduler/priority、blocking、queue/communication、GPU interference、multi-rate/sample-and-hold、phase 和 actuation bounds。

对任一事件或 loop window，必须分开保存：

1. Observed architectural correctness：这一次是否违反内部工程时间契约；
2. Observed physical correctness：这一次是否违反状态相关物理安全契约；
3. Analytical/formal timing guarantee：在已声明任务、调度、通信、阻塞和执行上界下，能否保证所有允许行为满足相应契约；
4. Single-run empirical headroom：仅根据本 run 有限样本得到的案例级经验余量。

若没有上述形式上界，Core 仍须完整给出 observed diagnosis；只是 Extension 状态为 NOT_ACTIVATED_MISSING_QUALIFIED_BOUNDS。不得让连续出现的 GUARANTEE_NOT_ESTABLISHED 淹没真正的 observed findings，也不得将本 run 的 max、P99 或拟合值称为 formal guarantee。它们只能称为：

本 run 观测窗口内的经验时间余量或经验尾部风险。

同一 run 中的连续帧还受共同状态和反馈影响，不是独立实验重复。

1.3 四类结果必须分开

每个关键字段必须属于以下之一：

结果类	含义	可用于什么
data/observed	直接测量，或由兼容直接观测确定性计算	本次实际行为与实际后果

结果类	含义	可用于什么
requirement/qualified	独立、前瞻、场景兼容且带不确定性边界的要求	判定本次是否违反时间契约
model/predicted	经声明模型得到的预测或反事实	模型能力、敏感性和反事实损失
retrospective/reconstructed	使用本 run 结果以后才知道的信息重建	事故重建与诊断，不能冒充前瞻要求

实际输入缺失时，data/observed 必须标记 UNAVAILABLE 并说明原因，禁止用模型预测补齐。

1.4 当前部署架构约束

当前系统中 Apollo 未向 Bridge 发送 Guardian 命令，Bridge 直接读取 Control。因此执行的命令链必须写为：

... -> Planning -> Control -> Bridge -> CARLA vehicle dynamics -> physical response

Guardian 可作为旁路观测或未执行的输出源，但不得代替 Control 出现在执行命令的主因果链中。

2. 双分析单位：Event-Centric 与 Loop-Centric

2.1 Run 只是一层容器

一个 run 可包含多个安全事件、多个目标、多个 sensor source、多个软件路径和多个物理 effect。若把 run 压缩为一个 E2E latency，会混合：

- 不同事件；
- 不同 source-to-sink path；
- 同一事件的首次反应与后续更新；
- 新数据产生的动作与旧数据复用产生的动作；
- 软件命令形成与车辆真正产生物理效果。

因此主键必须至少是：

run_id + event_id + dependency_bundle_id + chain_id + chain_instance_id

对于连续控制环，不能强行虚构一个 hazard 起点；使用第二套主键：

run_id + loop_id + window_id + mode_id + context_id

同一段数据可以同时属于一个事件单位和一个闭环窗口，但二者结论不得互相替代。例如某次障碍物响应可以在 U_E 中发生 Reaction miss，同时其所在的 U_L 窗口未表现出闭环振荡；反之，持续横向振荡可以在没有独立 PHYSICAL_CAUSE_EVENT 的情况下构成 Loop timing anomaly。

2.2 Safety-Relevant Event 的操作定义

Safety-Relevant Event 不是“日志里出现一行异常”，也不是碰撞、最小间距或迟迟未响应这类下游结果。它是物理世界或受监控系统状态中会启动或改变安全控制需求的离散变化。统一记录为：

```
E_k = {
  event_id, event_role=PHYSICAL_CAUSE_EVENT, event_type,
  target_id, scenario_geometry,
  cause_predicate, t_c, t_c_interval, clock_domain,
  state_at_cause, discovery_method, prospective_or_retrospective,
  source_evidence_ids, uncertainty, limitations
}
```

候选事件类型包括但不限于：

- 前车开始制动、静态障碍进入冲突走廊、行人/车辆切入；
- 交通灯状态改变、路权或限速约束改变；
- 横向路径冲突、车道边界/曲率突然收紧；

- 定位置信度或可用性降级并触发控制需求改变；
- 可独立确认会改变安全控制需求的系统能力/状态变化，例如定位可用性降级。

事件起点 t_c 定义为声明的 `cause_predicate`

首次持续成立的墙钟时刻或有界区间，而不是分析者事后随意挑选的某一帧。若采样分辨率只能界定区间 $[t_c^L, t_c^U]$ ，必须传播该不确定性。

以下对象只能先登记为 `DIAGNOSTIC_SEED` 或中间/结果事件，不能直接充当反应计时起点：Planning 首次 STOP、物理响应未形成、margin local minimum、接触、碰撞。它们可以向前搜索独立的 `PHYSICAL_CAUSE_EVENT`；若找不到独立 cause，则 Reaction contract 为 `NOT_TESTABLE_NO_INDEPENDENT_CAUSE`，不得以结果定义起点形成循环论证。若研究问题本身是“某内部状态变化到另一 effect 的时延”，必须另建事件并显式说明它不是物理 hazard reaction。

2.3 事件发现允许回看，但 deadline 构造不允许泄漏结果

单 run discovery 可以使用整段日志、碰撞和物理轨迹回看，生成需要分析的事件候选。这是 `RETROSPECTIVE_EVENT_DISCOVERY`，合法但必须标注。

一旦事件被选中，合格动态 deadline 的输入必须冻结在事件起点或规定 cut-off 之前：

```
event discovery may inspect the future
deadline construction may not use post-cause outcome
```

尤其不能用本 run 后续的实际停车距离、实际碰撞标签、实际 t_{phys} 或事后拟合制动能力来反推并充当该 run 的独立时间要求。

2.4 每个事件可能有多条 Cause-Effect Chain

若系统数据依赖图是 DAG，必须逐条 source-to-sink path 分析，而不是把所有 source 混成一个 latency：

```
LiDAR -> Fusion -> Prediction -> Planning -> Control -> Bridge -> physical effect
Radar -> Fusion -> Prediction -> Planning -> Control -> Bridge -> physical effect
Localization -> Planning -> Control -> Bridge -> physical effect
Chassis -> Control -> Bridge -> physical effect
```

ECRTS 2023 对多 source/sink DAG 采用逐 source-to-sink cause-effect path 的定义；Yi 等人的动态 deadline 也施加在 sensor-to-actuator path，而不是仅施加在单个 task 周期上。任务可调度不推出 E2E 路径必然满足 deadline。ECRTS 2023, Yi et al. 2021

逐 path 不等于把联合依赖拆成多个独立完整原因。对 Fusion/Prediction/Planning 的 AND-join、冗余或可选输入，另建：

```
dependency_bundle = {
  dependency_bundle_id, event_id, join_semantics,
  member_chain_ids,
  input_role = REQUIRED | CONTRIBUTING | REDUNDANT | OPTIONAL,
  completeness_rule, effect_predicate
}
```

只有满足 completeness_rule 的 bundle 才能声称该控制 effect 的联合输入链闭合。物理损失先在 event/effect 层计算一次，再按证据分配贡献状态；禁止把同一损失完整复制给每条输入 path 后相加。

2.5 事件与 effect 端点必须成对定义

框架不预设所有事件都以“制动开始”为 effect。对每个事件定义 effect predicate：

```
Psi_effect(event_type, target_id, vehicle_state, command_state) = TRUE/FALSE
```

可能的 effect

包括持续有效制动、转向响应、减速轨迹发布、紧急停车、降级模式进入或其他与事件相符的物理动作。至少保存：

- t_{cmd} ：首个与事件和目标因果相关、语义正确的 Control 命令；
- t_{apply} ：Bridge 实际接收/应用该命令；
- t_{phys} ：物理 effect predicate 首次持续成立；

- effect_hold_rule：持续时长、幅值、迟滞和去噪规则。

若 effect 只在软件层形成而物理响应不可观测，则软件 E2E 可以报告，但不得命名为 physical Reaction Time。

t_phys 不是分析者可自由调节的单值。每种 event/effect type 必须在当前 outcome 前登记

effect_predicate_spec：信号来源、方向、幅值阈值、持续时间、允许中断、滤波器、采样率、插值法和 hysteresis。主端点至少输出：

$$[t_phys^L, t_phys^C, t_phys^U]$$

并执行 Effect-threshold sensitivity audit。例如对制动起效同时评估预注册阈值组 $a < -0.3, -0.5, -0.8 \text{ m/s}^2$ 与不同 hold duration；这些组合不是看结果后挑一个最好看的端点，而是用于检验主结论对合理定义的稳定性。若端点变化足以改变 contract verdict，主结论必须降为 BOUNDARY_UNCERTAIN_EFFECT_DEFINITION；该区间继续传播到 T_R、slack、D_response、D_debt 和 physical-loss uncertainty。

2.6 Loop-Centric Analysis Unit

没有独立离散 cause 的连续控制问题使用：

```
loop_unit = {
  loop_id, controller, plant, feedback_sources, command_sink,
  window_id, window_start, window_end, window_selection_rule,
  mode_id, context_id, operating_state_bounds,
  sample_semantics, hold_semantics, physical_tracking_outputs,
  source_evidence_ids, uncertainty, limitations
}
```

窗口必须由预声明规则产生，例如固定时长滚动窗、每个 driving mode

的稳定段、曲率/速度区间或控制状态机驻留段。禁止围绕看到的最大 jitter

单点事后裁剪窗口后，把该窗口当作总体行为。状态跨界时切窗，不将不同 control mode、增益、速度、曲率或 target-load 条件混成一个分布。

Loop-Centric 的最低观测量是：

$$T_s[n] = t_sample[n] - t_sample[n-1], J_s[n] = T_s[n] - T_s^{ref}$$

$$T_feedback[n] = t_ctrl_read[n] - t_state_source[n], T_actuation[n] = t_phys_effect[n] - t_cmd_apply[n]$$

另保存 controller execution/activation、command update gap、trajectory/state reuse、sensor-to-controller coherence、command-to-feedback phase、tracking error、steering/acceleration rate 与 oscillation metrics。只有 timing anomaly 与物理控制行为在兼容时钟、同一 window/mode/context

下存在时序关联或经控制模型支持，才能写“闭环时间行为与跟踪退化相关”；单个 80 ms 周期尖峰不能自动证明控制失稳。

3. 数据与证据契约

3.1 数据源能证明什么、不能证明什么

数据源	主要可证明内容	不能单独证明的内容
Bridge/SCB 日志	请求和实际施加的 delay、范围、时间、消息数量、队列行为	下游传播、deadline miss、碰撞原因
Apollo 模块日志	决策、fallback、solver、目标语义、带标签阶段时刻	日志遗漏时的完整执行链；跨时钟严格差值
Trace events	模块内部 read/start/end/pub、队列/执行阶段、monotonic 时间	未锚定时不能直接和墙钟/跨主机相减
Trace message context	trace_id、parent、input/output seq、source data timestamp	target 语义正确性与物理 outcome
Parsed Cyber record	channel 写入/record receipt、header/source timestamp、消息 payload、发布率、reuse	每个模块内部真实 read/start/finish，除非消息字段或插桩显式提供
Localization/Chassis	带明确 timestamp 语义的速度、加速度、姿态与响应代理；可用于墙钟速度积分	Cyber 消息或状态估计不天然等同 CARLA ground truth；不能只凭它证明真实接触几何或软件原因
Collision event	接触时刻、actor、impulse 等直接碰撞事实	时间错误是碰撞原因
Actor history/ground truth	目标几何、间距、相对运动、CARLA frame/sim 诊断	不可用作 Apollo 当时已知状态，除非另有可用性证明

Apollo 官方文档说明，Record 用于记录 Cyber RT channel 上发送/接收的消息；因此 record timeline 是消息记录时间线，不等于模块内部调度时间线。[Apollo 10 Cyber RT Developer Tools](#)

3.2 时间戳字典

每个 timestamp

必须带：clock_domain、host、timestamp_type、unit、resolution、alignment_method、error_bound。至少区分：

- wall_epoch：墙钟 epoch；
- monotonic：单机 monotonic trace；
- record_time：消息被写入/观察到 record 的时刻；
- header_timestamp：模块发布 header 时间；
- measurement/source_time：传感器测量或被传播的数据源时间；
- CARLA_sim_time/frame：仿真推进时间和帧；
- Bridge_apply_wall：Bridge 应用命令的服务器墙钟。

任何差值运算前执行：

1. 单位核验；
2. 时钟域与主机核验；
3. offset/drift/anchor 估计；
4. 负 age、非单调和跳变检查；
5. 给差值附加误差区间。

P_CLOCK 必须按比较项判定，不存在“一次全局时钟通过后所有差值都可信”。同一墙钟上的 t_c 到 t_phys 可成立，而跨 Orin 与服务器的分段时间仍可能不可用。

3.3 时钟与 phase 必须分离

时钟对齐回答“两个 timestamp 能否相减”；phase 分析回答“producer、consumer、CARLA tick 和注入起点的相位是否造成采样/激活等待”。同步的时钟仍可能有 phase quantization，phase 相同也不能证明时钟无 offset。

分别输出：

- clock_alignment_audit.csv；
- phase_audit.csv。

没有主动 phase 扫描时，周期簇或 latency 台阶只能支持 PHASE_EFFECT_HYPOTHESIS，不能证明 phase 是根因。

3.4 证据分类与污染传播

关键证据至少采用：

```
DIRECT_OBSERVED
OBSERVED_DERIVED
TRACE_LINEAGE
WEAK_TEMPORAL_ALIGNMENT
INDEPENDENT_REQUIREMENT
INDEPENDENT_CALIBRATED_MODEL
VALIDATED_MODEL
UNVALIDATED_MODEL
RETROSPECTIVE_RECONSTRUCTION
OUTCOME_TRUNCATED
CONFLICTED_EVIDENCE
MISSING
```

若关键输入带 CLOCK_TAINT、TARGET_TAINT、MODEL_TAINT、RETRO_TAINT 或 OUTCOME_TRUNCATED，污染必须沿依赖链传播，不能因为下游公式是确定性的就被洗掉。

3.5 目标身份与功能正确性是独立前置条件

时间链必须和同一物理目标、同一决策语义绑定。建立：

- P_TARGET：sensor/Fusion/Prediction/Planning/Control/碰撞 actor 是否指向同一目标；
- P_FUNC：Perception 是否看见并持续跟踪目标、Prediction 语义是否有效、Planning 决策目标和位置是否合理、trajectory 是否有效、Control 是否收到正确 trajectory 并形成连续命令、Bridge 是否应用、物理响应是否出现。

只有 P_FUNC=QUALIFIED_PASS 且 C4_ARCH 或 C4_PHYS

在合格独立契约下成立，才能写“功能正确但时间错误”，并必须注明是工程时间违约还是安全关键时间违约。若 Planning 选错目标、无 STOP、trajectory infeasible 或 Control 命令语义错误，时间问题最多是多因素候选，不能把功能错误重新命名为实时性错误。

新增 P_BUNDLE、P_LOOP 与 P_EFFECT：

- P_BUNDLE 审计同一个 consumer job 实际消费的多源输入、join role、source epoch 和时间补偿；
- P_LOOP 审计 controller/plant/feedback 边界、window selection、mode/context 同质性与物理行为端点；
- P_EFFECT 审计 effect predicate 的来源、阈值、hold、filter、L/C/U 与敏感性。

三者分别限制 Coherence、Loop 和 physical Reaction 链的 claim ceiling，不能因 P_CLOCK/P_TARGET 已通过而自动通过。

3.6 时间污染与选择污染

在原有 CLOCK_TAINT、TARGET_TAINT、MODEL_TAINT、RETRO_TAINT 和 OUTCOME_TRUNCATED 之外，新增：

- BUNDLE_TAINT：join 成员或实际 consumed instance 只能由 nearest message 推断；
- COHERENCE_COMPENSATION_TAINT：共同 epoch 依赖未验证外推/补偿模型；
- WINDOW_SELECTION_TAINT：loop window 围绕异常事后选择或混合 mode/context；
- EFFECT_DEFINITION_TAINT：effect threshold/hold/filter 在看到 outcome 后选择或 sensitivity 不稳定；
- RESOURCE_ATTRIBUTION_TAINT：执行时间 residual 被未经资源证据地解释为 CPU/GPU/lock/scheduler 原因。

这些 taint 不因后续算术确定而消失，并沿相应 C3/C4/C5/C7 路径传播。

4. L3：重建 Event Chain 与 Loop Timing

在未知故障的 run 中，事件问题通常从 L3/L4 或 L6 seed 开始；连续控制问题可从 loop-window contract 或 physical tracking anomaly 开始。二者都不能先假设 L1 根因已知。

4.1 统一事件表

将日志、trace、record、Localization、Bridge 和碰撞数据映射为统一事件：

```
event_time.csv
{
  event_uid, run_id, event_id, chain_id, module, stage,
  event_kind, read_or_write, clock_domain, host,
  raw_time, aligned_wall_time, alignment_error,
  input_seq, output_seq, trace_id, parent_trace_id,
  data_source_time, target_id, payload_digest,
  source_file, source_locator, evidence_class, quality_flags
}
```

event_kind 至少覆盖：physical_cause、sensor_sample、receive/read、ready、start、end、publish/write、command、bridge_apply、physical_effect、outcome。

4.2 因果匹配优先级

Grade	匹配依据	允许的结论上限
A	显式 trace_id、parent/provenance、input/output seq 链	严格 cause-effect propagation 可支持
B	传播的 source/header timestamp + 已验证 downstream mapping	严格 propagation 可支持
C	同时钟、有界窗口、语义和顺序验证后的时间对齐	系统级时间关联/段级定位
D	nearest-time heuristic	不确定候选
UNKNOWN	证据不足	不可测试

禁止把各模块互不对应的 P99 相加成一条链；stage decomposition 必须来自同一个 causal chain instance。

4.3 Forward causal tracing

从事件的首个合格 source sample 开始，沿每条路径选择“最早读取了该 source 或其合法后继的数据”的 job：

source sample -> earliest eligible downstream read -> ... -> first qualifying command -> physical effect

同时检查：

- source data 是否被覆盖后从未到达 sink；
- 某 consumer 是否跳过该更新；
- 多速率链中实际消费的是哪一帧；
- 输出是否虽然发布，但与目标/事件语义无关；
- 首个 Control 命令是否真的被 Bridge 应用。

4.4 Backward provenance tracing

从实际 effect、Control 命令或 Planning 决策向前追溯其真正读取的数据：

```
physical effect <- applied command <- Control input trajectory
               <- Planning inputs <- Prediction/Fusion source samples
```

它回答“车辆动作基于哪一批世界状态”，用于计算 Data Age 和发现 reuse/staleness。Forward 与 backward 结果必须相互核验：

- forward sample 到达了哪个 effect；

- observed effect 实际基于哪个 sample ；
- 两者不一致时是否发生 overwrite、skip、reuse 或路径切换。

4.5 ECRTS partition 思想的工程化边界

ECRTS 2023 的 p-partitioned job chain 由立即 backward chain 与立即 forward chain 组成，其原始用途是证明 warm-up 后最大 MRT 与最大 MDA 的等价计算。将模块边界作为 partition point 可用于本框架的分段重建：

```
at partition p:
backward provenance from p to source
+ forward propagation from p to physical effect
```

但必须明确：

- 这是对论文结构的工程化扩展，不是论文已经证明的故障定位定理；
- 只有 read/write 事件、job 顺序、通信语义和 warm-up/valid-chain 条件被验证时，才可继承其严格 job-chain 含义；
- 缺少这些条件时，它只是 PARTITION_DIAGNOSTIC_VIEW，不能宣称严格 MRT/MDA 等价或唯一根因。

4.6 Multi-input Age Vector

Planning/Control 同时消费多个输入，不能只输出一个模糊的 Age：

$$A_c,k(t_e) = [A_perception, A_prediction, A_localization, A_chassis, A_map, \dots]$$

其中：

$$A_j^{read} = t_read - t_source, j, A_j^{cmd} = t_cmd - t_source, j, A_j^{apply} = t_apply - t_source, j, A_j^{phys} = t_phys - t_source, j$$

Age 必须连同评估端点命名，禁止使用含糊的 $t_effect/read$ 。每个分量保留 source semantics、chain、dependency_bundle_id、input_role、target relevance 和 clock error。A_critical 不是简单取最大值，而是从本事件控制决策所需的 REQUIRED/CONTRIBUTING 输入集合中按预声明的 join/completeness 规则取最不利且语义相关的分量；缺少依赖分析时只能报告 vector，不得擅自指定 critical source。

Age 分量分别合格，不代表多源 join 合格。对每次 bundle consumption instance q ，定义 source-time coherence：

$$K_B(q) = \max_j \text{ in } B_qt_source, j - \min_j \text{ in } B_qt_source, j$$

同时保存 pairwise skew matrix：

$$K_ij(q) = t_source, i - t_source, j$$

其中成员集合不能无条件包含所有可见 topic：仅纳入同一个 consumer job 实际读取、满足 bundle role/completeness 的输入；对异步 Fusion，要分别保存 measurement time、state-estimation epoch、prediction/compensation target epoch 和 consumer read time。若某模块把异步测量显式外推到共同 fusion epoch，必须同时报告：

```
raw_source_skew
compensated_state_epoch_skew
compensation_model/version/horizon
residual_time_alignment_uncertainty
```

不能因为 timestamp 被改写成共同 header 就宣称 coherence 已成立。Temporal Coherence requirement τ_C^{arch} 可以来自 Fusion/Planning 接口设计、传感同步规格或明确工程阈值； $\tau_C^{phys}(x)$ 则应由跨源时间偏斜导致的相对状态误差包络反解。没有合格阈值时只报告 COHERENCE_OBSERVED，不得写 INCOHERENCE_FAILURE。

4.7 Fresh Causal Update Gap

普通 output gap 只能说明模块有没有发布；实时安全更关心有没有新的、与事件相关的数据真正到达 effect。定义合格新鲜更新序列 q_n ：

```
qualifying(q_n) iff
target/event semantics match
AND source timestamp strictly advances
AND lineage reaches the selected sink/effect
AND output is not mere reuse of the same source state
```

则：

$$G^{fresh_n}=t(q_{n+1})-t(q_n)$$

必须同时报告：

- $G_{publish}$ ：发布间隔；
- $G_{source_advance}$ ：source timestamp 前进间隔；
- G_{fresh_effect} ：新鲜数据真正影响 sink/effect 的间隔。

因此“Control 100 Hz 输出”不能推出“系统 10 ms 获得一次新鲜安全决策”。Control 可能连续复用同一 Planning trajectory。

5. L2：局部时间原语与症状定位

5.1 每个 task/edge 的标准时间点

对相邻 producer i-1 与 consumer i，尽量记录：

t_{write_prev} producer 写出
 $t_{receive}$ 消息在 consumer/record 可见
 t_{ready} 对应 job 已具备运行条件
 t_{start} job 实际开始
 t_{end} job 处理结束
 t_{write} consumer 写出

在时钟兼容且语义明确时，分解：

$$\begin{aligned} L_i^{transport} &= t_{receive} - t_{write, prev} \\ L_i^{queue} &= t_{ready} - t_{receive}, L_i^{activation} = t_{start} - t_{ready} \\ L_i^{exec} &= t_{end} - t_{start}, L_i^{publish} = t_{write} - t_{end} \end{aligned}$$

若没有 ready，不得把 start-receive 强行解释成纯 queue 或纯 phase；应命名为 unresolved_wait。若 record 只有消息写入时间，则不能从 record 单独计算模块内部执行或调度等待。

5.2 标准局部症状集合

症状	操作量	典型机制候选
Sampling delay	$t_s - t_c$	sensor tick、phase、遮挡、触发策略
Transport delay	receive - producer write	网络、SHM/RTPS、bridge、序列化
Queue/backlog	ready - receive 或队列深度/积压	CPU/GPU contention、callback backlog
Activation/phase wait	start - ready 与周期相位	timer/callback phase、错过下一激活
Execution tail	end - start	算法路径、锁、资源竞争、fallback
Publish delay	write - end	序列化、锁、writer/backpressure
Staleness	consumer time - source time	缓存、排队、慢 producer
Reuse	多个 sink 输出引用同一 source/trajectory	sample-and-hold、上游 gap
Overwrite/skip	source update 无 downstream successor	多速率覆盖、buffer policy
Drop/reorder	seq 缺口/乱序且被直接证据支持	transport/queue policy
Actuation delay	physical effect - applied command	Bridge、底盘、动力学、控制建立时间

5.3 R/A/G/C/L 五维时间退化向量

局部与路径时间健康统一写为：

$$T_{deg}=[R,A,G,C,L]$$

- R：reaction/response-time behavior；

- A : data freshness/age ;
- G : update continuity/gap。
- C : 同一 multi-input bundle 的 source-time skew、pairwise skew 和补偿后 residual incoherence ;
- L : 闭环 sample interval/jitter、feedback delay、actuation delay、phase drift、reuse/hold 及其与控制行为的窗口级关系。

五者语义不同，不能互相补齐。对本 run 可报告 count、P50、P90、P95、P99、MAX 和 IQR/MAD，但必须写明这是 run 内相关样本分布，不是跨 run 可重复性或 formal bound。C 必须按 bundle/consumer instance 聚合；L 必须按 loop/window/mode/context 聚合，禁止将窗口内帧当成独立实验重复。

5.4 单 run 如何判定“退化”

L2 的“退化”必须有显式 reference。单 run 可合法采用：

- NOMINAL_PERIOD : 配置或设计周期；
- ENGINEERING_REQUIREMENT : 明确的模块/edge timing requirement；
- PRE_FAULT_WITHIN_RUN : 同 run、同工作状态下的故障前参考窗口；
- DECLARED_EXTERNAL_REFERENCE : 外部规格或独立校准参考；
- LOCAL_STATIONARY_REFERENCE : 经过状态分段验证后的同 run 稳态窗口。

若运行状态、算法分支、目标数量或计算负载发生变化，不能把前后窗口差异直接解释为纯时间退化。必须保存 context covariates，并将其列入 defeater。

无 reference 时允许写：

本 run 观测到 412 ms 的 Planning 发布间隔。

不允许写：

Planning continuity degraded。

5.5 症状不等于根因

Prediction -> Planning 等待 85 ms 只定位到一条 edge/segment。要把性质判成 phase、queue 或 scheduler interference，还需要相位、ready/start、调度/资源或队列证据。L2 输出应包含：

observed symptom
candidate mechanisms
supporting evidence
challenging evidence
missing discriminator

5.6 Loop-Centric 局部时间原语

对每个 U_L 至少保存：

类别	指标	语义边界
Sampling	T_s,J_s、missed/extra samples	区分 timer release、message arrival 与 controller execution
Feedback	source-to-read age、feedback delay、phase offset	必须使用实际被 controller 消费的 state instance
Controller	activation wait、execution、publish	缺 ready/start 时保留 unresolved wait
Hold/reuse	command hold、trajectory reuse、fresh command gap	高频重复发布不等于高频新控制信息
Actuation	command-to-apply、apply-to-response、deadband/build-up	软件命令与物理 effect 分开
Physical behavior	tracking error、overshoot、oscillation、rate/slew	是 outcome/behavior，不自动等于 timing cause

Loop anomaly 的 observed 判定优先依赖 Architectural Contract；若需判定“时间行为破坏闭环稳定性/安全”，还必须有控制器 delay/jitter margin、validated simulation/reachability envelope 或时间与物理行为的机制特异证据。仅凭相关性最多输出 LOOP_TIMING_ASSOCIATION。

因此 L 必须是结构化子向量，而不是把不同单位压成一个分数：

```
L^obs=[T_s,J_s,T_feedback,T_controller,G_cmd^fresh,T_actuation,_loop,H_reuse]
```

tracking error、overshoot 和 oscillation 是与 L 对齐的 physical/control behavior vector，不并入 L 后再循环证明。任何 composite loop verdict 都必须先逐子分量判定，再按预声明 required components 合成；缺失分量保持 NOT_TESTABLE。

6. L1 : Temporal Stressor 与根因候选

6.1 两种运行模式

Discovery mode : L1 未知。从 L4 miss、L6 loss 或强 L3/L2 anomaly 反向生成并检验候选。此时 L1 只保存 hypothesis 状态，不因“候选得到直接支持”就伪装成已确认 stressor entry。

Validation mode : L1

已知或主动注入。先验证故障签名真的进入闭环，再向下游验证传播；仍需检查替代解释，不能因“注入了 delay”就自动把碰撞归因于该 delay。

因此 Claim Ledger 中的 C1_STRESSOR_ENTRY 只回答“声明的 stressor/干预是否实际进入闭环”；内生机制候选的支持强度保存在 diagnosis hypothesis ledger，不能与 C1 用 OR 合并。

6.2 Temporal Fault/Stress Signature

无论发现还是验证，候选统一写成：

```
F_T = {
  fault_or_stressor_type, location, onset, end, duration,
  requested_magnitude, actual_magnitude, distribution,
  persistent_or_transient, one_shot_or_repeated,
  affected_channels/messages, queue_policy,
  drop_status, reorder_status, resource_context,
  direct_evidence, confidence
}
```

分类至少覆盖：

- sampling/trigger delay ;
- execution/WCET tail ;
- CPU/GPU/memory/lock contention ;
- scheduler/priority/blocking ;
- queue/backlog/backpressure ;
- network/transport/Bridge delay ;
- period/phase/mode-transition delay ;
- staleness/reuse/overwrite/drop/reorder ;
- actuation/control-build-up delay ;
- clock/timestamp artifact (替代解释，不是真实时间故障) ；
- functional or physical alternative (替代解释) 。

6.3 注入证据的边界

配置只能证明“意图注入”。只有 applied row、Bridge/SCB

实际延时或等价直接插桩才能证明干预进入声明位置。注入发生在 Bridge/Control

路径上时，它能证明系统对外部时间故障的响应，但不能单次证明 Apollo 自身存在内生实时缺陷。

若 fault onset 早于 t_c ，必须审计 $[t_f, t_c]$ 的 pre-hazard state divergence。事件起点的距离、速度、加速度、姿态可能已经是干预造成的中介/后处理状态，不能自动当作独立初始条件。

6.4 Resource Interference Graph

当 L2 将异常限制到 execution/unresolved-wait 时，L1 必须继续建立 Resource Interference Graph，而不能停在“Planning execution tail”。图节点至少包括 task/job、CPU runnable/running、mutex/futex、GPU submit/queue/kernel、memory/IO、network/SHM、logger/filesystem 与共址干扰任务；边表示 RUNS_ON、WAITS_FOR、BLOCKED_BY、PREEMPTED_BY、CONTENDS_WITH 或 SUBMITS_TO。

对一个 job 的观测时间尽量分解：

$$L_i^{job} = L_i^{CPU \setminus running} + L_i^{ready \setminus not \setminus running} + L_i^{blocked} + L_i^{GPU \setminus queue} + L_i^{GPU \setminus kernel} + L_i^{sync} + L_i^{IO} + L_i^{unknown}$$

每个分量必须来自 scheduler trace、thread state、lock/GPU instrumentation、resource counter 或等价直接证据；分解不完整时保留 residual，不把 wall execution - CPU time 一律命名为 scheduler delay。输出 resource_interference_graph.csv 和 resource_interference_edges.csv，并为每个机制候选给出：时间重叠、资源共享、优先级/阻塞关系、与异常 job 的时序先后、反例窗口和缺失 discriminator。没有资源证据时，结论停在 EXECUTION_SEGMENT_LOCALIZED。

7. L4 : Architectural 与 Physical 双层时间契约

L4 是整个框架的判定枢纽，但不是只有“物理安全 deadline”一条路。每个适用的 U_E 或 U_L 先评估 L4-A Architectural Temporal Contract，再独立评估 L4-B Physical Dynamic Contract。两者可以同时成立、只成立其一或都不可测试。

7.1 L4-A Architectural Temporal Contract

Architectural Contract 回答“系统是否违反自己的工程时间设计”，包括：

- task release/period/deadline 与 execution budget；
- producer-consumer edge age/freshness；
- module publish/update gap；
- source-to-sink E2E reaction/age；
- multi-input bundle coherence/source skew；
- sample/phase relationship、command hold/reuse；
- loop sampling jitter、feedback delay、actuation delay 与控制器允许的 delay/jitter margin。

要求注册为：

```
architectural_requirement = {
  requirement_id, owner, artifact, version, interface/task/path/bundle/loop scope,
  metric_semantics, value_or_bounds, unit, applicable_mode/context,
  timestamp_basis, aggregation_rule, miss_policy,
  effective_time, lock_time, provenance, qualification
}
```

合格来源优先级为：部署配置/接口契约/设计文档/经过确认的模块规范 > 独立测试标定的工程限制 >

明确标注的研究者分析阈值。仅从本 run 的中位数、最大值、默认日志频率或“Apollo 通常是 10 Hz”推断的阈值不得标记为 QUALIFIED_ARCHITECTURAL。若要求是 period，而观测量是 execution time 或 age，语义不兼容，不能直接比较。

Architectural miss 只证明 ARCHITECTURAL_TIMING_FAILURE，不自动证明物理危险。它可以作为 backward diagnosis seed，也可在离障碍物很远、无碰撞时成立。

7.2 L4-B Physical Dynamic Contract：先定义安全条件，再反解 deadline

设事件起点状态为 x_c ，响应延迟为 ρ ，模型/环境参数为 θ ，在分析时域 H 上定义安全裕量函数：

$$\Phi(x_c, \rho; \theta) = \min_{t \in [0, H]} h(x_{\text{ego}}(t), x_{\text{target}}(t), \text{geometry}, \text{policy})$$

$\Phi \geq 0$ 表示在声明的 dynamics、目标行为和 safety buffer 下不越过安全边界。动态反应 deadline 定义为：

$$\tau_{R,c,k}^{\text{req}} = \sup\{\rho \geq 0 \mid \text{for all } r \in [0, \rho], \inf_{\theta \in \Theta_m} \Phi(x_c, r; \theta) \geq 0\}$$

其中 Θ_m 是当前 context/微 ODD 的有界参数集合。这里取的是“从零延迟开始连续安全的前缀集合”；只有证明 Φ 对 ρ 的安全性单调时，才可简化为普通的最大安全根。若安全延迟集合不连通或单调性未知，必须报告 safe-delay intervals，并只把包含 0 的安全连通分量右端点作为响应 deadline。这样 deadline 是物理安全条件的逆解，而不是凭车速选择的经验常数。车速只是 x_c 的一个分量。

7.3 纵向跟驰/静态障碍的默认可解释实例

在同车道纵向场景，可使用经验证的 RSS-like 包络：

$$d_{\text{req}}(\rho) = v_{\text{erho}} + (1/(2)a_{\text{resprho}}^2 + \frac{(v_e + a_{\text{resprho}})^2}{2b_e} - (v_f^2)/(2b_f) + d_{\text{safe}})$$

求满足 $d_{\text{req}}(\rho) \leq d_{\text{clear}}$ 的最大非负 ρ 。静态障碍令 $v_f = 0$ ，不虚构目标制动收益。若当前已经不满足 $\rho = 0$ 的安全条件，则：

```
tau_req = 0
state = ALREADY_OUTSIDE_DECLARED_ENVELOPE
```

这表示安全包络在事件起点已经失守，不能再把后续所有损失简单归因于响应 delay。

Koopman、Osyk 与 Weast 指出，仅比较最终停车位置可能漏掉制动过程中的中途碰撞，因此 longitudinal model 必须检查 response period 和整个 braking trajectory 的 closest approach；摩擦、坡度、曲率、车辆和环境变化也会改变可用制动能力。Koopman et al. 2019

不适合纵向闭式公式的横穿、交叉口、曲线路径或多目标冲突，必须使用场景兼容的 reachability/trajectory model。若模型与几何不兼容，deadline 状态为 NOT_QUALIFIED_PRIMARY，不得硬套公式。

7.4 Physical Deadline 合格条件

动态 deadline 要成为本次 observed miss 的裁判，必须满足：

- state input 在 t_c 或声明 cut-off 前可得；
- target identity、geometry 和 target-motion assumption 明确；
- 车辆/目标 dynamics 与 braking envelope 来自外部规格或独立校准验证数据；
- calibration runs 与本次 evaluation run 不重叠；
- friction、grade、curvature、actuation build-up 和 uncertainty bounds 明确；
- 不使用本 run 的 post- t_c 停车、碰撞、 t_{phys} 或结果标签选择参数；
- 输出 $\tau_{\text{req_low/center/high}}$ 和完整 provenance。

不满足时分别归为 UNVALIDATED_MODEL、RETROSPECTIVE_RECONSTRUCTION 或 NOT_QUALIFIED_PRIMARY。

7.5 微 ODD / context uncertainty

真实系统不能假设精确知道 μ 、坡度、曲率和制动能力。将运行条件划分为参数有界的 context region m ：

$$\tau_{\text{robust}}(m) = \inf_{x \in X_m} \tau(x)$$

必须同时审计“当前状态是否真的落在所选 region 内”。微 ODD 方法把“region 内可分析的不确定性边界”和“是否选择了正确 region 的不确定性”分离，但不会让不确定性消失。Koopman et al. 2019

7.6 多维时间契约向量

Reaction、Freshness、Fresh Update Continuity、Temporal Coherence 和 Loop Timing 的含义不同。每个维度可同时有 architectural 与 physical 两类契约：

$$S_{\text{arch}} = \tau_{\text{arch}}^{\text{req}} - [R, A, G, C, L]^{\text{obs}}$$

$$S_{\text{phys}} = \tau_{\text{phys}}^{\text{req}}(x) - [R, A, G, C, L]^{\text{obs}}$$

其中各分量不能互借 threshold：

- Freshness contract 应通过“使用年龄为 a 的状态仍能在状态估计/预测误差包络内保证安全决策”反解；
- Fresh-update contract 应通过“在 g 时间内保持旧命令/旧轨迹时，reachable states 仍不越过安全边界”反解；
- Coherence contract 应通过接口同步要求或“跨源 skew 为 k 时，相对状态误差仍在融合/决策包络内”反解；
- Loop contract 应通过控制设计的 sampling/delay/jitter margin、验证过的闭环模型或明确工程限制产生；
- 缺少独立构造时，对应维度只能作为诊断量，不能借用 reaction deadline 或 nominal period。

ECRTS 的结论是 warm-up 后 最大 MRT 与最大 MDA 在其 job-chain 定义下等价，不表示同一事件的每个 observed Reaction 和 Age 数值相同，也不自动提供 A/G 的物理阈值。[ECRTS 2023](#)

契约判定必须按分量保存，不能因一个分量可测就替代其他分量：

```

verdict_arch_[R|A|G|C|L] = compare(observed, qualified_arch_requirement)
verdict_phys_[R|A|G|C|L] = compare(observed, qualified_physical_requirement)

architectural_contract_verdict / physical_contract_verdict:
  MISSED      if any applicable required component is CLEARLY_MISSED
  WITHIN      if every applicable required component is CLEARLY_WITHIN_CONTRACT
               and no required component is missing
  UNCERTAIN   if no component is missed and at least one is BOUNDARY_UNCERTAIN
  NOT_TESTABLE if no component is missed/uncertain and a required component is missing

```

required_components 必须由 event type、dependency bundle 和 safety model 预先声明。未适用分量用 NOT_APPLICABLE，不能与 NOT_TESTABLE 混淆。

任何 ARCHITECTURAL_MISSED + PHYSICAL_WITHIN 必须报告为“工程时间违约但本次尚未越过物理安全边界”；PHYSICAL_MISSED 则是 Safety-Critical Timing Failure，但仍不自动推出碰撞原因。

7.7 不确定性下的 observed contract verdict

设观测时间区间为 $[T_L, T_U]$ ，合格 deadline 为 $[\tau_L, \tau_U]$ ：

条件	判定
$T_U \leq \tau_L$	CLEARLY_WITHIN_CONTRACT
$T_L > \tau_U$	CLEARLY_MISSED
两区间重叠	BOUNDARY_UNCERTAIN
只有事后 deadline	RETROSPECTIVE_ONLY
只有未验证模型	MODEL_SUPPORTED_ONLY
输入/clock/target 缺失	NOT_TESTABLE

主 observed slack 建议同时保存保守、中心和乐观值，不把单一中心值包装成确定结论。

7.8 Observed correctness 与可选 Assurance/Guarantee Extension

本次观测：

$$S_{\text{obs}} = \tau_R^{\text{req}} - T_R^{\text{obs}}$$

分析保证：

$$S_{\text{guar}} = \tau_{R, \text{low}}^{\text{req}} - R_{\text{path}}^{\text{UB}}$$

其中 $R_{\text{path}}^{\text{UB}}$ 必须来自与部署系统相符的任务/调度/通信上界，包括 WCET/WCRT、release/period、priority/scheduler、blocking、queue/transport、multi-rate overwrite semantics、phase 和 actuation bound。

Yi 等人使用的路径近似 $\sum_i(p_i+r_i)$ 及其 $2\sum_i p_i$ 线性化依赖其单 CPU、周期任务、EDF、给定 WCET 和异步覆盖模型。它可启发 Apollo 的 budget 分解，但不能把 Apollo 一次 trace 中的均值或 period 直接代入后称为 formal guarantee。Yi et al. 2021

若 $S_{obs} \geq 0$ 但 $S_{guar} < 0$:

本次满足契约，但在声明模型下无法保证所有允许执行都满足该契约。

若没有合格 R^{UB} , $S_{guar} = \text{NOT_ESTABLISHED}$ 。可另列：

$\text{single_run_empirical_headroom} = \tau_{req} - \max \text{observed comparable instance}$

其中 comparable instance 必须限定为相同 event type、target semantics、dependency bundle/chain、mode、context region、effect predicate、端点定义和数据完整性等级。若候选由 retrospective seed 挑选，必须标注 selection bias；该值仍不得称为 guarantee。

Assurance Extension 不参与 Core 完成判定。只有 bound_assumption_audit=QUALIFIED 时才计算 S_{guar} 与 guarantee-loss points；否则只输出一行 extension 状态和缺失的 exact bounds，不在每个事件重复生成多个否定 verdict。

7.9 Dynamic deadline shrink 与 mode transition

Yi 等人指出，从宽松 deadline 切到更紧 deadline 时，旧模式的长周期 job 可能继续存在，使新 sensor data 产生额外等待；任务各自可调度仍不能保证过渡期 E2E deadline。Yi et al. 2021

本框架对 hazard window 内的 contract trajectory 计算：

$$d/dt S(t) = d/dt \tau^{req}(t) - d/dt T^{risk}(t)$$

当 $S > 0, d/dt S < 0$ 时，可报告局部线性诊断量：

$$T_{exhaust}^{local} = (S) / (-d/dt S)$$

它只是基于声明平滑窗口的短时外推，不能称为保证。还要检测 deadline region/mode change，在切换点单独核查旧 job、旧 trajectory 和新 contract 的共存时间。

7.10 Slack Waterfall 与 Guarantee-Loss Point

对 chain 边界 i ，定义观测前缀时间：

$$L_{prefix,i}^{obs} = t_i - t_c$$

观测剩余预算：

$$B_i^{obs} = \tau_R^{req} - L_{prefix,i}^{obs}$$

若存在从边界 i 到物理 effect 的合格 suffix bound $R_i \rightarrow \text{phys}^{UB}$ ：

$$B_i^{guar} = \tau_{R,low}^{req} - L_{prefix,i}^{obs}, U_{R_i \rightarrow \text{phys}^{UB}}$$

前缀与 suffix 只有在以下组合契约成立时才可相加：suffix 的起始边界状态、path/bundle、mode、phase、backlog、clock basis、资源/调度假设与该 observed prefix 兼容；suffix 不得重复包含已经计入前缀的 wait/exec；运行时假设必须有审计结果。否则该边界 GUARANTEE_NOT_COMPOSABLE。逐边界输出：sampling、transport、queue/unresolved wait、activation、execution、publish、consumer wait、command-to-apply、actuation。

必须区分三个时刻：

1. GUARANTEE_NOT_ESTABLISHED_FROM_START：分析入口处

$B_0^{guar} < 0$ ，说明从起点就未建立该保证，不能称为“后来丢失”；

2. FIRST_CONDITIONAL_GUARANTEE_LOSS：入口处非负且之后首个 $B_i^{guar} < 0$ 的边界，显示名为“首次条件性证明余量为负的边界”；后续实际历史使允许未来集合缩小时，可以建立新的条件性保证，但这不是原始允许行为集合上的同一保证恢复；

3. FIRST_IRREVERSIBLE_OBSERVED_MISS：首个满足 $L_{prefix,i}^{obs}, L > \tau_U$ 的边界；即使后续 effect 立即发生，本次 miss 也已不可挽回；

4. OBSERVED_EFFECT_MISS : t_phys 到来后最终确认的 miss。

observed_contract_miss_time 也必须是区间：当累计 elapsed-time 下界首次超过 tau_U 时给出确定 miss 下界；若只在 t_phys 才能确认，则其区间由 endpoint/alignment uncertainty 传播。FIRST_CONDITIONAL_GUARANTEE_LOSS 只说明“从这里起，已有前缀 + 合格后缀上界无法保证完成”，不等于根因就在该模块。输出固定附带：

```
causal_interpretation = NOT_INFERRED
root_cause_status = REQUIRES_BACKWARD_DIAGNOSIS
```

根因定位还需检查哪个 segment 相对本地 budget/reference 发生超额消耗。

若没有 suffix bound，只能画 observed slack waterfall，不得输出 guarantee-loss point。

8. 双向诊断：从失效 seed 回溯到机制

8.1 诊断 seed 优先级

优先级从强到弱：

1. 合格 L4-B physical contract 明确 miss；
2. 合格 L4-A architectural contract 明确 miss；
3. L6 直接安全损失，L4 尚不可测试；
4. L3 的 R/A/G/C 或 loop-window timing/physical behavior 异常；
5. L2 具有 reference 的局部退化；
6. 无 reference 的 case-level anomaly。

下游 seed 只能触发上游假设生成，不能倒推出 C1 已证实。

8.2 Backward diagnosis 算法

对每个 seed 执行：

1. 冻结 seed：明确事件、目标、chain、时间区间、契约和 outcome；
2. 定位 lineage/loop segment：事件单位由 effect/command 向 source 回溯；闭环单位沿 feedback→controller→command→plant→feedback 周期定位首个证据中断或 contract 异常边；
3. 枚举候选：sampling、execution、queue、activation/phase、transport、staleness/reuse、coherence skew、overwrite/drop、mode transition、feedback/actuation、resource interference；
4. 加入替代解释：clock artifact、target mismatch、functional failure、initial unsafe state、braking/road/geometry、outcome conflict；
5. 时间可达性检查：候选 onset 必须早于 seed，并存在执行架构上的可行传播路径；
6. 证据三分：每个候选列 supporting、challenging/contradicting、missing evidence；
7. 观测等价类：把当前数据无法区分的候选保留在同一 equivalence class；
8. 给出 discriminator：准确指出需要哪个已有字段、哪一 trace 点或哪一调度/资源证据才能区分；
9. 限制结论强度：只能在替代候选被反驳/界定后升级。

8.3 根因状态

```
SUPPORTED_CANDIDATE
CONSISTENT_BUT_UNRESOLVED
CHALLENGED
REFUTED
NOT_TESTABLE
NOT_APPLICABLE
```

诊断强度：

强度	含义
DETECTED	下游时间/物理失效直接成立
LOCALIZED_TO_SEGMENT	因果链和预算把问题限制在某段
MECHANISM_SUPPORTED	有机制特异证据，且主要等价候选被界定
ISOLATED_CANDIDATE	仅一个非反驳候选剩余
INTRINSIC_DEFECT_SUPPORTED	内部机制、可重复性及外部/功能/物理替代解释均得到处理

单 run 出现唯一“最大耗时段”通常最多支持 LOCALIZED_TO_SEGMENT，不能自动升级到 ISOLATED_CANDIDATE。

8.4 Forward validation

当存在已知注入或已定位机制时，再沿正向链检验：

```
verified stressor
-> matching local primitive changes
-> causally matched chain timing effect
-> qualified contract state
-> physical budget loss
-> outcome
```

只有正向证据闭合，才能把 backward candidate 升级为更强的归因结论。

9. L5 : Physical Safety Budget Loss

9.1 Canonical observed response distance

对物理 cause t_c 到物理 effect t_{phys} ，主响应距离统一使用车辆速度对墙钟时间的端点插值梯形积分：

$$D_{\text{response,wall}}^{\text{data/observed}} = \int_{t_c}^{t_{phys}} v(t) dt_{\text{wall}}$$

字段名：

`D_response_wall_integral_data_observed_m`

兼容旧字段时可保留 `D_delay_wall_integral_data_observed_m`，但它只能是同值别名。CARLA frame/sim time、Localization 空间位移和 realtime factor 是独立诊断量，不能替代该主指标。

9.2 Deadline-excess distance debt

合格 deadline endpoint 是区间而不是默认中心值：

$$[t_d^L, t_d^U] = [t_c^L + \tau_L, t_c^U + \tau_U]$$

若本次 miss，对每个明确声明的 deadline 选择 q in {center, high}：

$$D_{\text{debt}, q}^{\text{requirement} \setminus \text{constrained}} = \max(0, \int_{t_c}^{t_{phys}} v(t) dt_{\text{wall}})$$

主表至少输出 `D_debt_req_low/center/high_m`，并另给由 t_c, t_{phys} 、clock alignment、速度测量与积分覆盖传播得到的数值区间。积分要求：端点线性插值；wall timestamp 严格单调并去重；按与道路前进方向一致的非负 longitudinal speed 积分（倒车/符号改变必须另建规则）；超过预声明 gap threshold 的区间标 INCOMPLETE_COVERAGE 而不跨缺口插值；保存 sample count、coverage ratio、最大 gap、单位和数值容差。它表示“超过合格响应期限以后，物理响应仍未形成期间继续占用的道路距离”，不是“若及时响应就一定能少损失的反事实距离”。其 ancestry 是“合格 requirement + observed velocity path”，应标记 REQUIREMENT_CONstrained_DERIVED，不能命名为纯直接观测。

- model deadline -> `D_debt_model_predicted_m`；
- retrospective deadline -> `D_debt_retro_diagnostic_m`；
- 无合格 deadline -> 主 `D_debt` 不可用。

9.3 观测空间预算

对静态纵向完整停车：

$$\begin{aligned} M_0^{\text{data/observed}} &= D_1 - D_{\text{response}}^{\text{obs}} - D_{\text{brake}}^{\text{obs}} \\ M_{\text{safe}}^{\text{requirement} \setminus \text{constrained}} &= M_0^{\text{obs}} - D_{\text{safe}}^{\text{req}} \end{aligned}$$

其中 D_1 、 D_{response} 、 D_{brake} 必须端点兼容。若 fault onset 早于 t_c ，分别报告：

- Total Closed-loop Effect：包含 t_f 到 outcome 的全部状态演化；
- Post-cause Response Effect：只分析 t_c 以后的响应链。

不要在同一预算式中同时减去总 response distance 和其内部的 deadline-excess debt，避免重复计数。

9.4 全轨迹物理安全裕量

对移动目标或复杂几何，先保存不带政策阈值的实际几何/接触量：

$$C_{\min}^{\text{data/observed}} = \min_{t \in [t_c, t_o]} \text{clearance}(x_e(t), x_o(t), \text{geometry})$$

其中 t_o 是 outcome/采集完整终点， x_o 是经 target-identity 审计的对方 actor 状态；两者连同 measurement source、measurement time、transport age、geometry/shape 定义和轨迹 coverage 保存。这可以使用 CARLA ground truth/actor history 做实际 outcome 评估，但必须与 Apollo 当时“知道什么”分开。

若引入安全阈值/包络，另算 requirement-constrained margin：

$$M_{\text{phys}}^{\text{requirement} \setminus \text{constrained}} = \min_{t \in [t_c, t_o]} [\text{clearance}_{\text{obs}}(t) - d_{\text{safe}}^{\text{req}}(t)]$$

这里 $d_{\text{safe}}^{\text{req}}$ 的来源、单位、锁定时间和适用域必须明确。若 h 还含模型预测或 reachable-set envelope，则结果归入 model/predicted，不能写成纯 observed。对于模型安全包络：

$$M_{\text{phys}}^{\text{model/predicted}(\rho)} = \min_t h_{\text{model}}(x_e(t; \rho), x_o(t), \Theta_m)$$

时间 slack 与物理 margin 是同一安全边界的不同坐标，但观测 margin 和模型 margin 不能混在同一列。

9.5 “造成多少损失”的五个互补答案

单 run 必须分别报告：

1. $D_{\text{response_wall_integral_data_observed_m}}$ ：本次响应形成前实际走了多远；
2. $D_{\text{debt_requirement_constrained_m}}$ ：超过合格 deadline 后响应仍未形成期间继续占用多少道路距离；
3. $\text{minimum_clearance_data_observed_m} + \text{contact/collision}$ ：本次实际几何与接触 outcome；
4. $\text{safety_margin_requirement_constrained_derived_m}$ ：实际 clearance 减去合格安全阈值后的裕量；
5. $\Delta M_{\text{phys_model_predicted_m}}$ ：模型中相对声明反事实响应 ρ_{ref} 的裕量损失：

$$\Delta M_{\text{phys}}^{\text{model}} = M_{\text{phys}}^{\text{model}}(\rho_{\text{ref}}) - M_{\text{phys}}^{\text{model}}(T_R^{\text{obs}})$$

第 5 项是反事实模型结果，不是直接观测。没有第二个现实世界反事实轨迹时，不能把它写成“直接观测到 timing 导致 X 米裕量损失”。

式中 ρ_{ref} 必须在本次 outcome 之前由 requirement、设计基线或独立对照模型锁定，并保存 $\rho_{\text{ref_source}}$ 、 $\rho_{\text{ref_lock_time}}$ 和语义（例如 zero-delay、deadline-compliant response 或 nominal controller response）。看完本 run 后挑选使损失最大的 ρ_{ref} 无效。

9.6 Collision right-censoring

碰撞会截断后续无碰撞停车轨迹。碰撞 run 可直接报告：

- collision、actor、time/frame、impact speed/impulse；
- 碰撞前 observed response/braking segment；
- 碰撞时 actual clearance/contact state。

但必须将“完整 observed braking distance、完整停止位置和 full-stop margin”标为 UNAVAILABLE_OUTCOME_TRUNCATED，不得用模型停车距离填成 observed。

10. L6 : Physical Safety State

L6 只负责物理后果，不负责反推时间原因。优先保存连续量：

- minimum/final clearance ；
- physical/contact/engineering margin ；
- minimum speed、strict stop、near stop ；
- impact speed、impulse、collision actor ；
- truncated braking time/distance ；
- safety envelope violation duration/area (若定义明确) 。

离散 outcome 可采用：

```
SAFE
SAFE_LOW_MARGIN
CRITICAL_MARGIN
MINIMUM_DISTANCE_VIOLATION
EMERGENCY_MANEUVER
NEAR_MISS
CONTACT
COLLISION
HIGH_SEVERITY_COLLISION
UNCERTAIN
```

除 CONTACT/COLLISION 等直接事件外，low/critical/near/high-severity 阈值必须有外部、预注册或明确工程定义的 provenance，不能看完结果后挑阈值。

因此 OUTCOME_OBSERVED 与 MARGIN_CLASSIFICATION 必须分开：碰撞可直接确立不安全 outcome；“裕量退化/低裕量”只有相对合格 requirement/reference 才成立。单 run 的连续 clearance 变化若无阈值或参考，只能描述，不能命名为 degraded。

无碰撞文件不等于安全停车；还需确认采集窗口完整、目标身份、最终速度/间距与传感器状态。

11. 单 run 完整执行流程与两个执行档位

11.1 Core Profile : 每个 run 强制执行

Core Profile 是 Observed Diagnostic Core，不要求形式上界。它至少完成：输入/时钟审计、event catalog、loop catalog、lineage、R/A/G/C/L primitives、Architectural/Physical contract qualification、effect sensitivity、物理 outcome 与精简 Claim–Evidence summary。

11.2 Extended Profile : 按异常与证据资格触发

以下任一条件触发 Extended Profile：合格工程/物理 contract miss；强物理 outcome；持续 loop anomaly；需要从 execution segment 深挖资源机制；存在 qualified path/suffix bound；或用户明确要求证明/归因。Extended 增加 full defeater ledger、Resource Interference Graph、formal/suffix bounds、guarantee-loss waterfall、微 ODD robustness、counterfactual model 和详细 mechanism isolation。

Core COMPLETED + Extended NOT_TRIGGERED/NOT_QUALIFIED 是合法完整结果，不得写成“分析未完成”。

Step 0 : 冻结输入与只读原始数据

- 建立 run inventory：配置、collect window、日志、trace、collision、actor history、record、Bridge/SCB；
- 原始 run 目录只读；生成物写到独立 analysis directory；

- 明确本 run 是否有 record，禁止把相邻时间戳的其他 run record 关联进来。

Step 1：建立数据可用性与时钟审计

- profile 每个数据源的覆盖窗口、行数、parse error、缺失 channel；
- 建立 timestamp dictionary、clock alignment 与 phase audit；
- 任何不兼容差值先停在 NOT_TESTABLE，不进行“看起来合理”的相减。

Step 2：分离诊断 seed 与物理 cause 事件

- 从 margin local minimum、response-not-formed、contact/collision、L4/L3 异常和已知注入生成 diagnostic_seed；
- 对每个 seed 向前查找独立的 physical conflict/control-demand change predicate，只有它可定义 t_c ；
- 每个 cause event 声明 target、geometry、effect predicate、 t_c 区间和 discovery provenance；
- 相邻对象按目标与因果关系去重/关联，但保留 seed IDs 和 cause-event IDs；找不到独立 cause 时 L4 Reaction verdict 为 NOT_TESTABLE_NO_INDEPENDENT_CAUSE。

同时建立 loop catalog：枚举 longitudinal、lateral、localization-feedback、trajectory-tracking 等实际闭环，按 mode/context 切分预声明 window；Loop-Centric 不要求虚构 t_c 。

Step 3：限定目标与功能链

- 审计 P_TARGET 和 P_FUNC；
- 错目标、功能失效、fallback/infeasibility 保持为独立替代解释；
- 确认执行架构中 Guardian 不在 Bridge 实际命令链。

Step 4：为每个事件枚举依赖束与实际可行 paths

- 从 DAG/config/trace/record channel 生成 dependency bundles 和 source-to-effect paths，声明 join semantics 与 input role；
- 对每条 path 做 forward causal tracing 与 backward provenance；
- 输出 lineage grade 和无法穿透的边界。

对每次 multi-input join 保存实际读取成员、source epoch、pairwise skew、raw/compensated coherence 和补偿 provenance；不能从“topic 最近一条消息”代替实际 consumed inputs。

Step 5：计算 L3 observed chain metrics

- $T_{\text{sampling}} = t_s - t_c$ ；
- $T_{\text{software}} = t_{\text{cmd}} - t_s$ ；
- $T_{\text{actuation}} = t_{\text{phys}} - t_{\text{cmd}}$ 或进一步分成 command-to-apply + apply-to-physical；
- $T_R = t_{\text{phys}} - t_c$ ；
- Age Vector、publish/source-advance/fresh-effect gaps、bundle coherence；
- 所有端点与 uncertainty 写入表中。

并按 loop/window/mode/context 计算 sample interval/jitter、feedback age/delay、controller timing、command gap/reuse、actuation delay、phase drift 及物理 tracking/oscillation 指标。

Step 6：提取 L2 局部原语

- 仅对同一 chain instance 分解 transport/wait/activation/exec/publish/reuse；
- 与合法单-run reference 比较；

- 无 reference 的异常只做描述；
- 输出 local symptom 和候选机制，不提前宣称根因。

若 execution/unresolved wait 成为异常段，Core 先标记 RESOURCE_EVIDENCE_REQUIRED；Extended 再建立 Resource Interference Graph，禁止用 wall duration 猜 CPU/GPU/lock 原因。

Step 7：独立注册 L4-A 与构造 L4-B contract

- 从部署配置、接口/设计文档和已确认规范注册 Architectural Contract，核验 scope、metric semantics、mode/context 与 lock time；
- 选择场景兼容 physical safety model；
- 冻结 cut-off 前 state 与独立 dynamics/uncertainty；
- 对 R/A/G/C/L 分别注册工程阈值；可用时独立构造物理 $\tau_R, \tau_A, \tau_G, \tau_C, \tau_L$ 的 low/center/high；
- 运行 anti-leakage 和 domain qualification；
- 不合格时保留 model/retrospective 结果，但主 observed contract verdict 为不可测试。

Step 8：先判定 observed correctness，再决定是否激活 guarantee extension

- 分别给 R/A/G/C/L 的 architectural/physical uncertainty-aware verdict，再按预声明 required components 合成 event/loop verdict；
- 执行 effect-threshold sensitivity，将端点不确定性传播到所有下游量；
- 若有合格 suffix/WCRT bounds，计算 slack waterfall 和 guarantee-loss points；
- 若无 formal bound，仅输出 observed waterfall、single-run empirical headroom 和一条 ASSURANCE_EXTENSION_NOT_ACTIVATED 状态；
- 对 shrinking contract 检查 mode-transition/old-job/old-trajectory 共存。

Step 9：Backward diagnosis 到 L1

- 从 miss/loss seed 反向定位 segment；
- 建立 hypothesis ledger、equivalence classes、defeaters 与 discriminators；
- 对 resource candidate 使用 Resource Interference Graph；
- 根因表述强度不超过证据上限。

Step 10：计算 L5/L6

- 从 observed wall-clock velocity 重算 D_{response} ；
- 有合格 deadline 才计算主 D_{debt} ；
- 计算实际全轨迹 margin/outcome；
- 模型反事实与 observed 分表；
- collision run 应用 right-censoring。

Step 11：归因与结论生成

- 逐项检查 clock、phase、target、function、initial state、braking、geometry、freshness 和 outcome conflict defeaters；
- 从 Evidence Ledger -> Claim Ledger -> Defeater Ledger 生成结论；
- 报告最强合法语言和仍未证明的部分。

12. 数据源到分析量的具体映射

12.1 日志

可提取：

- Perception target 出现/消失、tracking continuity；
- Prediction target/trajectory；
- Planning STOP/decision/fallback/infeasibility/latency debug；
- Control command/latency；
- Localization speed/acceleration/pose；
- Bridge/SCB requested/applied delay；
- collision and actor history。

日志 timestamp 的语义必须逐模块确认；文本出现顺序不自动等于执行顺序。

12.2 Trace 插桩

当前 e2e_trace_v3 类数据可包含：

```
events: event_id, trace_id, module, phase, mono_ns, pid, tid
message_context: edge, channel, input_seq, output_seq,
                 data_ts_ns, primary_parent_trace_id,
                 parent_trace_count, trace_valid
```

这类字段适合：

- Grade A/B lineage；
- 模块内部 proc/runonce/substage timing；
- source timestamp 传播；
- parent/fusion lineage；
- overwrite/skip 的链路核对。

但 monotonic trace 要通过 trace anchor 映射到墙钟，映射残差进入 uncertainty。

12.3 Parsed Cyber record

本工作区已有历史 record 解析样本，可验证一次完整解析通常提供：

- record_message_index、channel coverage/rate/gap；
- Localization/Chassis state；
- Planning header、task latency、decision、trajectory；
- Control command、latency、vehicle state、Planning reuse/age；
- Perception/Prediction obstacles 与 source headers；
- sensor timeline；
- Planning-Control alignment、sensor-to-Control message diagnostics。

重要限制：

- 只看得到被录入的 channel 和 protobuf 字段；
- record_time 是记录/接收时间，不等于 task start；
- sensor_to_control_reaction_time 是消息级 source-to-Control 诊断，不能替代 t_c 到 t_{phys} 的 physical Reaction Time；

- obstacle table 可能按 obstacle row 展开，不能直接用它判断空帧发布 gap；应使用完整 channel timeline；
- Planning total_time_ms 可能在每个 task row 重复，frame statistic 必须按 planning_seq 去重。

若被分析 run 没有 record：

- 不跨 run 借用 record；
- record-only 可得的 Age/reuse/message payload 字段保持不可用；
- 使用 trace/log 可支持的部分继续分析；
- L3 lineage 与 L2 mechanism 的结论强度按缺失证据降级，而不是用猜测补齐。

12.4 Collision 与物理轨迹

- collision event/JSONL：直接 outcome；
- actor history：目标身份、相对运动、几何和 realtime factor 诊断；
- Localization wall-time velocity：主响应距离积分；
- CARLA frame/sim time：仅独立报告 e2e_sim_frames、realtime_factor 等诊断字段。

13. 必备输出文件

实际工具采用 Core/Extended 两级产物。Core 文件不可因字段缺失而省略；应保留 UNAVAILABLE + missing_reason。Extended 文件只在触发或用户要求时生成，并由 analysis_profile_status.csv 说明资格和触发原因。

```
analysis/<run_id>/
├── report/
│   └── single_run_realtime_diagnosis.md
├── tables/
│   ├── input_inventory.csv
│   ├── analysis_profile_status.csv
│   ├── event_catalog.csv
│   ├── event_timeline.csv
│   ├── loop_catalog.csv
│   ├── loop_windows.csv
│   ├── loop_timing_metrics.csv
│   ├── loop_physical_behavior.csv
│   ├── chain_catalog.csv
│   ├── chain_instances.csv
│   ├── dependency_bundles.csv
│   ├── partition_lineage.csv
│   ├── local_timing_primitives.csv
│   ├── age_vector.csv
│   ├── temporal_coherence.csv
│   ├── fresh_update_gaps.csv
│   ├── effect_predicate_registry.csv
│   ├── effect_threshold_sensitivity.csv
│   ├── temporal_fault_signature.csv
│   ├── clock_alignment_audit.csv
│   ├── phase_audit.csv
│   ├── target_identity_audit.csv
│   ├── functional_correctness_audit.csv
│   ├── requirement_registry.csv
│   ├── architectural_contracts.csv
│   ├── dynamic_contract_construction.csv
│   ├── contract_verdicts.csv
│   ├── velocity_trajectory_observed.csv
│
│   ├── physical_budget_observed.csv
│   ├── physical_outcomes.csv
│   ├── l5_recomputation.csv
│   ├── evidence_ledger.csv
│   ├── claim_ledger.csv
│   ├── claim_edges.csv
│   ├── claim_evidence_summary.csv
│   ├── exclusions_and_missing.csv
└── extended/
```

- slack_waterfall.csv
- guarantee_loss_points.csv
- timing_model_registry.csv
- path_and_suffix_bounds.csv
- bound_assumption_audit.csv
- resource_interference_graph.csv
- resource_interference_edges.csv
- physical_budget_model_predicted.csv
- diagnosis_hypothesis_ledger.csv
- diagnosis_edges.csv
- defeater_ledger.csv
- counterfactual_model_audit.csv
- validation/
 - data_quality_audit.md
 - anti_leakage_audit.md
 - arithmetic_recomputation.md
 - claim_audit.md
 - validation.json

13.1 每事件与每闭环窗口最终报告模板

Event / Target / Geometry

Cause predicate and t_c interval

Selected critical chain(s) and lineage grade

Observed chain timing

T_{sampling} / T_{software} / $T_{\text{command_to_apply}}$ / $T_{\text{actuation}}$ / T_R

Age Vector / $A_{\text{critical basis}}$

G_{publish} / $G_{\text{source_advance}}$ / $G_{\text{fresh_effect}}$

raw / compensated bundle coherence + pairwise source skew

Loop timing (when applicable)

loop / window / mode / context

T_s / J_s / feedback delay / controller timing / command hold-reuse / actuation

physical tracking error / overshoot / oscillation

Architectural contracts

R/A/G/C/L requirement IDs, provenance, scope and qualification

per-component and composite observed verdict

Physical dynamic contracts

τ_R low/center/high + qualification

τ_A / τ_G / τ_C / τ_L if independently available

per-component and composite observed verdict

Effect endpoint

predicate spec and t_{phys} L/C/U

threshold/hold/filter sensitivity and verdict stability

Assurance extension (only when activated)

formal/analytical guarantee status and S_{guar}

empirical headroom, explicitly non-guarantee

Slack waterfall

per-segment observed budget use

guarantee-not-established-from-start or first conditional proof-margin-negative boundary

first irreversible observed miss

dominant over-budget segment relative to declared reference

Diagnosis

localized segment

mechanism candidates and evidence

resource interference subgraph when execution/wait is implicated

observational equivalence class

competing functional/clock/phase/physical explanations

diagnosis strength and missing discriminator

Physical propagation

$D_{\text{response_wall_integral_data_observed_m}}$

$D_{\text{debt_requirement_constrained_m}}$

minimum_clearance_data_observed_m / contact / collision

safety_margin_requirement_constrained_derived_m

$\Delta M_{\text{phys_model_predicted_m}}$

right-censoring status

Outcome and allowed conclusion

13.2 新增核心表的最小字段契约

表	最小字段
analysis_profile_status.csv	run_id,core_status,extended_trigger,extended_status,qualification_gap,notes
loop_catalog.csv	run_id,loop_id,controller,plant,feedback_sources,command_sink,sample_semantics,h old_semantics,source_evidence_ids
loop_windows.csv	loop_id>window_id,start_wall,end_wall,selection_rule,selected_before_outcome,mode _id,context_id,context_bounds,homogeneity_status,taint_tags
loop_timing_metrics.csv	loop_id>window_id,instance_id,metric_name,value,unit,source_event_id,effect_event_i d,clock_domain,error_bound,reference_id,verdict,missing_reason
loop_physical_behavior.csv	loop_id>window_id,behavior_metric,value,unit,endpoint_definition,reference_id,verdict, lineage_to_timing_grade
temporal_coherence.csv	run_id,event_or_window_id,bundle_id,consumer_instance_id,input_id,input_role,sourc e_time,source_semantics,read_time,raw_skew_ms,compensated_epoch,compensate d_skew_ms,compensation_model,clock_error_ms,requirement_id,verdict,lineage_gra de,missing_reason
effect_predicate_registry.csv	effect_spec_id,event_type,signal,comparison,threshold,hold_ms,hysteresis,filter,samp le_basis,lock_time,provenance,qualification
effect_threshold_sensitivity.csv	event_id,effect_spec_id,threshold,hold_ms,filter,t_phys_low,t_phys_center,t_phys_hig h,T_R_low,T_R_center,T_R_high,contract_verdict,verdict_stability
architectural_contracts.csv	requirement_id,owner,artifact,version,scope_type,scope_id,dimension,metric_seman tics,low,center,high,unit,mode_context,effective_time,lock_time,evidence_class,qualific ation
resource_interference_graph.csv	node_id,node_type,host,process,thread,resource,instance_scope,start,end,evidence_ ids,availability
resource_interference_edges.csv	source_node,target_node,relation,overlap_ms,priority_or_blocking_detail,evidence_id s,status,residual_uncertainty

所有表均须保留 source_file/source_locator 或可解析的 source_evidence_ids。missing_reason 不能以模型值补齐；reference_id/requirement_id 必须能回链到 requirement registry；所有跨源 skew 与 loop 差值都继承 P_CLOCK 的 error budget。

14. Claim–Evidence 判定门

每个核心主张采用 I-M-E-C-O-N：

- I / Inputs：范围、数据和前置主张；
- M / Metrics：字段、单位、时钟、端点和聚合范围；
- E / Evidence：允许的证据类、支持/挑战证据和污染；
- C / Criterion：可证伪判据、不确定性和缺失行为；
- O / Output：verdict、confidence ceiling、允许语言和残余不确定性；
- N / Next：进入下一主张所需的准确条件。

在核心主张前先评估独立前置门：

前置门	强通过所需证据
P_CLOCK	相关差值的时钟域、anchor/sync、offset/drift/residual、resolution 与 error budget 已界定
P_PHASE	相关 producer/consumer/tick 的 period、phase origin 与 phase sensitivity 已界定；只在相应主张需要时作为前置
P_TARGET	event/chain/outcome 的物理目标身份一致
P_FUNC	与结论相关的感知、预测、规划、控制、Bridge 与物理响应语义已审计

前置门	强通过所需证据
P_BUNDLE	multi-input join 的成员、role、实际 consumed instance、source epoch 与 compensation provenance 闭合
P_LOOP	loop、plant/controller/feedback、window selection、mode/context 与物理行为端点定义合格
P_ARCH_CONTRACT	工程要求的 owner/artifact/version/scope/metric/unit/mode/context/lock time 可审计且与观测语义兼容
P_PHYS_CONTRACT	物理要求前瞻、独立、场景兼容、无 outcome leakage，并有完整 uncertainty/domain audit
P_EFFECT	effect predicate 在 outcome 前锁定或作为既定方法复用，L/C/U 与 threshold sensitivity 已传播

P_ARCH_CONTRACT 与 P_PHYS_CONTRACT 不得互相替代；P_BUNDLE 不通过时，Coherence 只能描述；P_LOOP 不通过时，Loop 结论最多是未限定窗口的 case-level observation；P_EFFECT 不稳定时，依赖 t_phys 的 C3_EVENT/C4_PHYS/C5 必须降级。

14.1 核心主张

ID	主张
C1_STRESSOR_ENTRY	声明的 stressor/故障/干预实际进入闭环；只适用于有外生声明或可直接观测 entry 的情形，内生机制候选不得通过 OR 升级本门
C2	R/A/G/C/L 时间行为相对声明 reference 发生退化
C3_EVENT	退化沿相关事件因果链传播到命令/物理 effect
C3_LOOP	loop window 的 sampling/feedback/actuation 时间行为与闭环物理行为在同 mode/context 下得到支持；相关性与机制因果分级保存
C4_ARCH	observed timing 明确违反合格 Architectural Temporal Contract
C4_PHYS	observed timing 明确违反合格 Physical Dynamic Contract
G1_GUARANTEE	Assurance Extension 中分析/形式上界是否建立、保持或出现条件性证明余量为负；独立于 C4_ARCH/C4_PHYS，未激活不影响 Core 完整性
E1_EMPIRICAL	single-run empirical headroom；仅描述性，不参与 guarantee/causal claim 门
C5	C4_PHYS=CLEARLY_MISSED 后形成 requirement-constrained physical budget consumption；C4_ARCH 或 G1 loss 不能单独触发
C6_OUTCOME	直接观测到 contact/collision/clearance 等物理 outcome；若声明 margin degraded，还需合格 threshold/reference
C7	处理替代解释后，时间失效对物理退化的归因达到声明强度

C7 是归因审计，不是第七层。

14.2 结论强度

Level	可合法表述
0	观测/测量到某个时间或物理事实
1	相对明确 reference 观测到时间退化
2	event cause-effect propagation、loop timing association 或段级系统关联得到支持
3A	在合格 architectural requirement 下，工程时间违约得到支持
3P	在合格 physical requirement 下，安全关键时间违约得到支持
3G	在合格模型下，formal/analytical guarantee status 得到支持；不自动进入物理归因链
4	timing 对物理安全退化的实质贡献得到支持
5	timing-dominated safety-failure candidate
6	在声明域内支持 SUT-intrinsic temporal defect

最弱关键证据/前置条件决定整体上限，不取平均。

14.3 默认 defeaters

clock alignment uncertainty
 phase/tick quantization
 target mismatch
 functional failure or fallback
 initial unsafe state
 pre-cause treatment-induced state divergence
 braking capability / friction / grade / curvature mismatch
 geometry or actor association conflict
 record/trace window incomplete
 outcome truncation
 deadline leakage or model-domain mismatch
 freshness/reuse alternative
 cross-source temporal incoherence or compensation error
 loop-mode/context mixing or post-selected window
 effect predicate / threshold / hold-rule instability
 resource-interference observational equivalence
 external injected fault versus intrinsic Apollo defect

关键 defeater 为 OPEN/UNKNOWN 时，受影响主张至多 UNCERTAIN/PARTIAL_PASS。

15. 结果分类：如何准确回答核心问题

15.1 哪里出了问题？

按可支持强度输出：

CHAIN_IDENTIFIED
 SEGMENT_LOCALIZED
 TASK_OR_EDGE_LOCALIZED
 MECHANISM_SUPPORTED
 ROOT_CAUSE_ISOLATED
 NOT_LOCALIZABLE

“最大耗时 task”不是自动答案；需要相对预算/reference 的 over-consumption 和时间可达的因果路径。

15.2 它是什么性质？

从局部原语和机制证据分类：

SAMPLING
 TRANSPORT
 QUEUE_BACKLOG
 ACTIVATION_PHASE
 EXECUTION
 PUBLISH_BACKPRESSURE
 STALE_DATA
 REUSE_SAMPLE_HOLD
 OVERWRITE_SKIP
 DROP_REORDER
 MODE_TRANSITION
 BRIDGE_DELAY
 ACTUATION
 CLOCK_ARTIFACT
 FUNCTIONAL_ALTERNATIVE
 PHYSICAL_ALTERNATIVE
 UNRESOLVED_MIXTURE

15.3 什么时候失去时间正确性或保证？

必须返回一个或多个不同语义的答案：

- architectural_contract_miss_time/window；
- physical_contract_miss_time/window；
- observed_contract_miss_time_interval，并说明由哪个下界首次越过哪个 deadline 上界；
- Assurance Extension 激活时的 guarantee_not_established_from_start；

- Assurance Extension 激活时的 first_conditional_proof_margin_negative_boundary/time (machine code 可保留 FIRST_CONDITIONAL_GUARANTEE_LOSS) ；
- first_irreversible_observed_miss_time ；
- contract_shrink_crossing_time ；
- 若无 formal bound ，只在 extension summary 明确 ASSURANCE_EXTENSION_NOT_ACTIVATED ，不把它误写为 observed contract 未知。

15.4 为什么失去？

返回：

```
supported mechanism
+ causal segment
+ consumed budget relative to reference
+ evidence IDs
+ counter-evidence
+ unresolved equivalent candidates
+ missing discriminator
```

不能只返回自然语言猜测。

15.5 造成多少物理安全损失？

返回五元组并保留 provenance：

```
L_physical= (D_response^obs,D_debt^req,C_min^obs,M_safe^req\ derived,Delta M_phys^model)
```

任何不可用分量明确说明原因。

16. 禁止的推理与常见歧义

1. collision -> deadline miss：错误；碰撞可由功能、物理或初始状态解释。
2. deadline miss -> collision：错误；可能仍有充足空间或安全 fallback。
3. output exists -> function correct：错误；目标、语义和轨迹可能错误。
4. record_time difference -> execution time：错误；record 不天然给 task start/end。
5. Control publishes at 100 Hz -> fresh control every 10 ms：错误；可能复用旧 Planning。
6. MRT maximum = MDA maximum -> every RT equals every Age：错误；论文结论作用于定义和最大值，并有 warm-up/chain 条件。
7. p-partition point -> root cause proven：错误；它是分段重建/计算结构，根因仍需机制证据。
8. single-run P99/max -> formal guarantee：错误；只能作为本 run 经验尾部描述。
9. first guarantee-loss point -> fault origin：错误；它是契约状态边界，不是唯一机制起点。
10. model deadline -> observed deadline miss：错误；未验证模型只能支持模型结论。
11. same-run stopping distance -> independent deadline：错误；属于事后重建。
12. CARLA sim frame displacement -> D_response：错误；主距离必须是墙钟速度积分。
13. collision-truncated trajectory -> full observed stopping distance：错误；右删失。
14. Bridge injection -> Apollo intrinsic defect：错误；最多支持外部故障下的系统响应与安全候选。
15. D_response + D_debt both subtracted：错误；debt 是 response distance 的子区间，会重复计数。
16. diagnostic seed -> physical cause/t_c：错误；下游 response-not-formed、margin minimum、contact/collision 只能触发 cause 搜索。

17. guarantee loss -> C5 physical loss : 错误；本次 observed contract 未 miss 时，分析保证丢失不能单独证明已发生 timing-caused physical budget loss。
18. one input path -> whole fusion cause : 错误；联合输入必须满足 dependency-bundle completeness，物理损失也不能逐 path 重复相加。
19. clearance - safety threshold -> pure observed margin : 错误；结果至少是 requirement-constrained derived；含预测包络时属于 model/predicted。
20. every input age within limit -> bundle coherent : 错误；多源 source-time skew 必须单独评估。
21. same header/fusion epoch -> temporal coherence : 错误；还需审计原始 measurement time、补偿模型和 residual uncertainty。
22. single loop jitter spike -> closed-loop instability : 错误；需窗口、mode/context、工程/控制 margin 与物理行为证据。
23. architectural miss -> physical safety miss : 错误；工程违约可以在物理裕量仍充分时成立。
24. no physical deadline -> no real-time problem : 错误；合格 Architectural Contract 可以独立判定工程时间错误。
25. wall execution tail -> CPU contention : 错误；需 Resource Interference Graph 的资源/调度证据。
26. choose effect threshold after outcome -> stable t_phys : 错误；主 predicate 必须预声明并做敏感性传播。
27. missing WCRT -> observed diagnosis incomplete : 错误；仅表示 Assurance Extension 未激活。

17. 方法完整性与单 run 可回答边界

问题	最低证据	缺失时的合法结果
发生了什么物理 event？	ground truth/target/事件 predicate	EVENT_UNCERTAIN
哪条 chain 处理了它？	trace/provenance/headers	ASSOCIATION_ONLY / NOT_TESTABLE
哪里耗时？	read/start/end/write 或可解释边界时间	UNRESOLVED_SEGMENT
多输入是否时间一致？	同一 consumer instance 的 source epochs + qualified coherence reference	COHERENCE_OBSERVED / NOT_TESTABLE
连续闭环是否时间异常？	loop/window/mode/context + sampling/feedback/actuation + reference	LOOP_OBSERVATION_ONLY / NOT_TESTABLE
是否违反工程时间设计？	observed compatible metric + qualified architectural contract	ARCH_REQUIREMENT_MISSING / NOT_TESTABLE
是否越过物理安全时间？	observed timing + qualified physical deadline/margin	MODEL_ONLY / RETROSPECTIVE_ONLY / NOT_TESTABLE
是否失去保证？	qualified path/suffix upper bounds；仅 Extended	ASSURANCE_EXTENSION_NOT_ACTIVATED
为什么失去？	机制特异证据 + alternatives	CANDIDATE_SET_ONLY
损失多少空间？	wall velocity + endpoints + qualified deadline	RESPONSE_DISTANCE_ONLY / UNAVAILABLE
实际安全后果？	collision/geometry/complete trajectory	OUTCOME_UNCERTAIN
是否 Apollo 内生缺陷？	内部机制、隔离、重复性、替代解释	NOT_ESTABLISHED_FROM_SINGLE_RUN

NOT_TESTABLE 是完整而诚实的分析结果，不是方法失败。框架的目标是把证据边界说清楚，而不是强迫每个 run 形成一条六层完整故事。

18. 每节逻辑审查清单

在写入最终报告前，逐事件执行：

事件与链

- event predicate 是否可复现，t_c 是否有误差区间？
- target 是否一致？一事件是否错误混合多目标？
- 每条 chain 是否来自同一 causal instance？
- forward 与 backward lineage 是否一致？warm-up/overwrite 是否处理？

时间语义

- Reaction、Age、Gap、Coherence、Loop 是否保持不同语义？
- Coherence 是否基于同一 consumer 实际读取的 bundle，而非各 topic nearest row？
- Loop window 是否按 mode/context 切分且未围绕 outlier 事后选择？
- software command 与 physical effect 是否分开？
- record/header/source/trace/wall/sim time 是否分开？
- clock error 是否传播到 slack 和 verdict？

Contract

- Architectural 与 Physical Contract 是否分表、分 scope、分 verdict？
- Architectural threshold 是否来自可审计 artifact，而非本 run 分布反推？
- physical deadline 是否从物理安全条件逆解？
- 参数是否在 cut-off 前锁定并独立于本 run outcome？
- geometry/model/domain 是否兼容？
- low/center/high 是否完整？
- observed correctness、formal guarantee、empirical headroom 是否分开？
- Effect predicate 是否预注册并输出 L/C/U 与 sensitivity？

定位与根因

- guarantee-loss point 是否被误写为 root cause？
- 本地 overrun 是否有 reference/budget？
- phase/queue/execution 是否有足够 discriminator？
- execution 候选是否通过 Resource Interference Graph 深挖，未知 residual 是否保留？
- backward hypothesis 是否被错误升级为 forward proof？

物理传播

- D_response 是否按墙钟速度梯形积分并覆盖端点？
- D_debt 是否使用合格 deadline？
- moving target 是否检查全轨迹 closest approach？
- observed 与 model 是否分表？
- collision truncation 与重复计数是否处理？

结论

- 是否列出支持证据、挑战证据、defeaters 和残余不确定性？

- 语言是否超过最弱关键链路允许的 claim level ?
- 是否明确回答哪里、性质、何时、为什么、损失和后果 ?

19. 三篇论文的融合方式与不可直接移植部分

19.1 ECRTS 2023

吸收：cause-effect chain 的 read/write job 语义、immediate forward/backward chain、MRT/MDA

区分与最大值等价、partition 计算思想、warm-up 与多路径逐 path 分析。具体依据是论文第 3 节的 MRT/MDA job-chain 定义、第 4 节的 p-partitioned job chains，以及第 5 节 Theorem 14 的最大 MRT/MDA 等价结论；其工程含义讨论见第 6 节。

本版扩展边界：论文在多 source/sink DAG 上逐 source-to-sink path 分析；本框架额外引入 dependency-bundle coherence 来处理自动驾驶 AND-join 的跨源时间一致性。这是工程扩展，不是论文已有 theorem。

不直接移植：MRT=MDA 不等于单事件 Reaction=Age；partition theorem 不等于 root-cause localization theorem；逐 path timing 不自动证明 multi-input bundle coherence；缺少完整 job read/write 时不能宣称严格定理已实例化。

19.2 Yi et al. 2021

吸收：每个 sensor arrival 的动态 E2E deadline、逐 source-to-sink path constraint、task schedulability 与 E2E guarantee 分离、shrinking deadline/mode-transition 的额外等待风险。由此本版进一步明确 Architectural Contract 与 Physical Dynamic Contract 是不同判定层，Observed Core 与 Formal Assurance Extension 是不同执行档位。具体见论文第 II 节系统模型与动态 E2E deadline 定义、第 III 节 Eq. (16) 的逐路径 $2\sum p_i$ 约束，以及第 IV 节的 relaxing/shrinking deadline mode-change protocol。

不直接移植：论文以能耗优化、单 CPU、周期任务、EDF 和给定 WCET 为模型；其 $2\sum p_i$ 近似不能直接当作 Apollo/Cyber RT 的实际 bound；车速—固定行驶距离公式只是特定应用示例，不是通用物理 deadline。

19.3 Koopman, Osyk & Weast 2019

吸收：context-dependent safety envelope、response time 物理含义、全 braking trajectory closest approach、friction/slope/curvature/vehicle/environment variability、微 ODD 有界参数思想。具体见论文第 3.1 节 RSS following-distance/response-time 定义、第 4 节及 Eqs. (2)–(4) 的响应期与制动全程 closest-approach edge cases，以及第 5.1–5.2 节的 μ ODD 分区和边界选择不确定性。

不直接移植：RSS-like 纵向公式不能覆盖所有自动驾驶场景；微 ODD 选择仍有不确定性；论文不提供本实验车辆的制动参数，参数仍需独立标定/验证。

19.4 本框架新增而非论文定理的工程诊断量

以下定义是本框架为单 run 诊断提出的工程量，不应引用为三篇论文已证明的结论：

- Event-Centric/Loop-Centric 双分析单元；
- [R,A,G,C,L] 五维时间正确性及 bundle coherence；
- Architectural/Physical 双层 contract 与 Core/Extended profiles；
- effect-threshold sensitivity 与 Resource Interference Graph；
- $\tau_{\text{robust}}(m)$ 的本实验 context-region 实例化；
- slack derivative dot S 与 $T_{\text{exhaust_local}}$ ；
- conditional guarantee-loss waterfall 与首次条件性证明余量为负的境界；
- requirement-constrained distance debt 与 Claim–Evidence 门。

它们是否有效取决于本文声明的模型、组合契约、时钟/端点质量和 evidence gates，而不是仅凭论文引用自动成立。

20. 最终框架的最简表达

对每个安全事件—联合依赖束—因果链实例，以及每个控制环—窗口—模式—情境：

$$\{ (U_E, U_L) \rightarrow (R, A, G, C, L)^{obs} \parallel (\tau_{arch}^{req}, \tau_{phys}^{req}(x)) \}$$

先由部署设计产生 Architectural Contract，由物理状态和独立安全模型产生 Physical Dynamic Contract，再由 event chain 与 loop window 数据产生实际时间行为：

$$\{ S_{arch} = \tau_{arch}^{req} - T^{obs}, S_{phys} = \tau_{phys}^{req}(x) - T^{obs} \}$$

若 Assurance Extension 已激活且有合格上界：

$$\{ S_{guar} = \tau_{R, low}^{req} - R_{path}^{UB} \}$$

Core 先定位工程/物理 observed miss、coherence/loop anomalies 与局部症状；Extended 才沿 chain 展开 guarantee slack waterfall，定位“从起点未建立保证”、首次条件性证明余量为负和实际不可逆 miss。再向后以 L2 primitives 与 Resource Interference Graph 检验 L1 机制候选，向前计算：

$$\{ D_{response}^{obs}, D_{debt}^{req}, C_{min}^{obs}, M_{safe}^{req} \setminus derived, \Delta M_{phys}^{model}, Outcome^{obs} \}$$

因此，最终结论不再是：

加了 300 ms，车辆撞了。

而是：

对事件单位 U_E 与闭环单位 U_L ，本次 observed Reaction、Freshness、Continuity、Cross-Source Coherence 和 Loop Timing 是否违反合格工程契约；是否进一步越过物理动态契约；若 Assurance Extension 资格满足，形式保证何时失去；哪些局部时间原语、资源干扰与机制证据解释了预算消耗；这种时间失效目前只是工程设计违约、闭环风险，还是已经造成 requirement-constrained 物理安全预算损失；以及结论的证据上限是什么。

参考文献与官方资料

1. M. Günzel, H. Teper, K.-H. Chen, G. von der Brüggen, J.-J. Chen, “On the Equivalence of Maximum Reaction Time and Maximum Data Age for Cause-Effect Chains,” ECRTS 2023. <https://doi.org/10.4230/LIPIcs.ECRTS.2023.10>
2. S. Yi, T.-W. Kim, J.-C. Kim, N. Dutt, “Energy-Efficient Adaptive System Reconfiguration for Dynamic Deadlines in Autonomous Driving,” 2021. <https://arxiv.org/abs/2106.04508>
3. P. Koopman, B. Osyk, J. Weast, “Autonomous Vehicles Meet the Physical World: RSS, Variability, Uncertainty, and Proving Safety,” SAFECOMP expanded preprint, 2019. <https://arxiv.org/abs/1911.01207>
4. Apollo 10.0, “Apollo Cyber RT Developer Tools.” https://apollo.baidu.com/docs/apollo/10.x/md_cyber_2docs_2cyber__developer__tools.html
5. S. Shalev-Shwartz, S. Shammah, A. Shashua, “On a Formal Model of Safe and Scalable Self-driving Cars.” <https://arxiv.org/abs/1708.06374>